Explanation: Reproduction Work of Alzheimer's Disease Diagnosis Based on Discrete Ricci Flow and Shannon Entropy

I will follow the mathematical process outlined in the paper, combined with my Python code, to explain as clearly as possible the principles and implementation of each step. The goal is to provide a clear view of the complete transformation process from raw 3D brain images to the final classification features, and to show how the code faithfully embodies these mathematical operations.

1. Problem Background and Overall Pipeline

- **Task**: Utilize the 3D shape differences of the hippocampus to distinguish between Alzheimer's disease (AD) patients and cognitively normal (CN) individuals.
- **Core Idea**: Conformally map the hippocampal surface to a plane using discrete Ricci flow, then extract two geometric quantities — the **conformal factor** λ and the **mean curvature** H — and compute their **Shannon entropy**, finally classifying with XGBoost.

The overall pipeline is illustrated in Fig. 3 of the paper:

1. Reconstruct the hippocampal triangular mesh from MRI.
2. Optimize the metric using discrete Ricci flow to obtain a planar parameterization.
3. Compute λ and H at each vertex.
4. Calculate the Shannon entropy of λ and H, forming the feature vector $[\mathrm{Entropy}(\lambda), \mathrm{Entropy}(H)]$.
5. Classify using XGBoost.

Below, I focus on explaining the mathematical principles and code implementation for steps 2, 3, and 4.

2. Discrete Ricci Flow: Conformal Mapping from 3D Surface to 2D Plane

2.1 Why Ricci Flow?

We want to map the 3D surface (hippocampus) **conformally** to a plane. Conformal means that **angles are preserved** after mapping, while areas may stretch or shrink. Such a mapping is determined by a **metric**, which can be adjusted by changing the scale factor e^{u_i} at each vertex. The goal of the adjustment is to make the curvature after mapping satisfy the planar conditions: zero curvature at interior points, and total curvature of 2π at boundary points (Gauss-Bonnet theorem).

> Regarding conformality:
> How does a discrete mesh truly guarantee "conformality"?
> Discrete Ricci flow introduces the vertex circle intersection angle ϕ_{ij}. Suppose we draw a circle at each vertex v_i and v_j (with radii γ_i and γ_j), and the angle between the tangents at their intersection is ϕ_{ij}. This angle remains invariant under discrete conformal deformation.

> There is a theorem: In discrete geometry, as long as we strictly preserve the intersection angles between adjacent circles during deformation and only change the radii γ of the circles, such deformation is mathematically proven to be conformal.
> Reflected in the paper and code: the variable we optimize, u_i, is essentially the logarithm of the vertex circle radius ($u_i = \log(\gamma_i)$). The optimization algorithm can only modify this scaling factor u; it cannot alter the underlying circle intersection structure. When u changes, causing radii to scale, the physical edge lengths necessarily change to maintain the intersection angles, thereby altering triangle angles θ and curvature K. This ensures the mapping is conformal.

2.2 Discrete Setup and Notation

- The mesh has V vertices and F triangular faces.
- Each vertex v_i has a **conformal factor** u_i, defining the radius $r_i = e^{u_i}$ (circle packing radius).
- The length of edge e_{ij} scales under conformal transformation as:

$$l_{ij}^{\text{new}} = l_{ij}^{\text{initial}} \cdot e^{u_i + u_j}. \tag{1}$$

This is the core formula implemented in the `_update_metric()` method.

2.3 Discrete Gaussian Curvature

The discrete Gaussian curvature at vertex v_i is defined as the **angle deficit**:

$$K_i = \begin{cases} 2\pi - \sum \theta_i^{jk}, & \text{interior vertex}, \\ \pi - \sum \theta_i^{jk}, & \text{boundary vertex}, \end{cases}$$

where $\sum \theta_i^{jk}$ is the sum of the angles at vertex i over all triangles incident to i. The interior angles are obtained from edge lengths using the law of cosines.

In the code, during each iteration of `run_flow()` :

- Compute edge lengths from current u (`_update_metric`).
- Call `compute_face_angles` to calculate the three angles of each triangle.
- Accumulate angles to each vertex using `np.add.at` to obtain `angle_sum` .
- Compute current curvature `K_current` by subtracting `angle_sum` from 2π or π, depending on whether the vertex is on the boundary (`boundary_indices`).

2.4 Ricci Flow Equation and Ricci Energy

The continuous form of Ricci flow is:

$$\frac{du_i}{dt} = \bar{K}_i - K_i, \tag{2}$$

where $\bar{K}_i$ is the **target curvature** (0 for interior vertices, distributed among boundary vertices such that the total sum is 2π).
This equation arises from minimizing the **Ricci energy**:

$$E_M(u) = \int_{(0,\ldots,0)}^{(u_1,\ldots,u_n)} \sum_{i=1}^{n} (\bar{K}_i - K_i)\, du_i. \tag{3}$$

E_M is a convex function, and its gradient is exactly $\bar{K}_i - K_i$. Thus, **minimizing E_M is equivalent to solving $\bar{K}_i = K_i$.**

Note: The code does not explicitly compute E_M; it directly uses its gradient for gradient descent updates of u, because we only care about the gradient direction.

Here a slight simplification is made: in the iteration, gradient descent is chosen instead of Newton's method mentioned in the paper. Although the original algorithm uses Newton's method for solving, gradient descent (explicit Euler discretization) is simpler and more intuitive in the code. As long as the step size is reasonable, it can stably converge to the target curvature.

Code update:

```
error = K_bar - K_current
self.u += step_size * error          # corresponds to explicit Euler discretization of (2)
self.u -= np.mean(self.u)            # remove global scale ambiguity
```

2.5 Setting Target Curvature on the Boundary

For a topological disk (hippocampus region), the total target curvature should be 2π, distributed among boundary vertices. In the code, it is simply averaged:

```
K_bar = np.zeros(self.num_vertices)
if len(self.boundary_indices) > 0:
    K_bar[self.boundary_indices] = (2 * np.pi) / len(self.boundary_indices)
```

A more accurate approach would distribute based on exterior angles, but uniform averaging is sufficient.

3. Planar Embedding

After optimization, we obtain a new set of edge lengths l_{ij}^{final} (computed from final u using Eq. (1)). Now we need to actually lay out these edges on the plane to obtain 2D coordinates for each vertex.

3.1 Basic Idea: Incremental Triangle Laying

We start with an arbitrary triangle and place it in the plane (e.g., first vertex at origin, second on the x-axis). Then we traverse all triangles via BFS: for each already positioned edge, find the adjacent triangle sharing that edge, and determine the coordinates of the third vertex using the two known vertices and the two edge lengths.

3.2 Computing New Vertex Coordinates

Let the known edge be AB with length d_{AB}, and the unknown vertex C such that $AC = l_{AC}$, $BC = l_{BC}$. From the law of cosines, the angle α at A satisfies:

$$\cos\alpha = \frac{l_{AC}^2 + d_{AB}^2 - l_{BC}^2}{2 l_{AC} d_{AB}}, \quad \sin\alpha = \sqrt{1 - \cos^2\alpha}. \tag{4}$$

Let $\vec{u}$ be the unit vector from A to B. Then the direction of $\vec{AC}$ is obtained by rotating $\vec{u}$ by α (assuming counter-clockwise orientation of the triangle):

$$\vec{v} = (\cos\alpha\, u_x - \sin\alpha\, u_y,\ \sin\alpha\, u_x + \cos\alpha\, u_y).$$

Then $C = A + l_{AC}\vec{v}$.

Code implementation:

```
cos_alpha = (len_ac**2 + d_ab**2 - len_bc**2) / (2 * len_ac * d_ab)
cos_alpha = np.clip(cos_alpha, -1.0, 1.0)
sin_alpha = np.sqrt(1 - cos_alpha**2)
...
vec_ac = np.array([Rx, Ry]) * len_ac
uv[v_c] = pos_a + vec_ac
```

Here `Rx, Ry` are the components of the rotated unit vector.

3.3 Why Is This Embedding Possible?

Because the optimized metric ensures that the angle sum around each interior vertex is close to 2π, coordinates will be consistent when arriving at the same vertex via different paths (satisfying the compatibility condition). This is exactly the guarantee of Ricci flow convergence.

4. Feature Extraction

After obtaining the planar embedding, we compute two geometric features at each vertex: the **conformal factor** λ and the **mean curvature** H.

4.1 Conformal Factor λ (Equation (5) in the paper)

The conformal factor describes the **local area scaling** from the 3D surface to the 2D plane. The paper defines it as:

$$\lambda(\nu) = \sum_{f \in B} \frac{\log(\text{area}_f^{3D}) - \log(\text{area}_f^{2D})}{2}, \tag{5}$$

where B is the one-ring neighborhood of vertex ν.

Code implementation:

- Compute area of each face in the 3D mesh `areas_3d` .
- Compute area of each face after 2D embedding `areas_2d` (zero-pad 2D coordinates to 3D and call the same area function).
- For each face: `face_lambda = 0.5 * (np.log(areas_3d) - np.log(areas_2d))` .
- Average the face values to its three vertices: accumulate and divide by the number of incident faces.

4.2 Mean Curvature H (Equation (6) in the paper)

Mean curvature describes the bending of the surface in the normal direction. The discrete form is:

$$H(\nu) = \sum_{e \in B} \frac{l(e)\,\beta(e)}{\text{area}(B)}, \tag{6}$$

where $l(e)$ is the edge length, $\beta(e)$ is the **dihedral angle** at edge e (angle between the normals of its two adjacent faces), and $\text{area}(B)$ is the area of the one-ring neighborhood of the vertex.

Code implementation:

- Build an edge-to-face mapping `edge_map` .
- Compute the unit normal vector of each face.
- For each interior edge (with 2 adjacent faces):
  - Compute the dihedral angle $\beta = \arccos(\mathbf{n}_1 \cdot \mathbf{n}_2)$.
  - Edge length `length` .
  - Product `val = length * beta` , accumulate to the two endpoints in `sum_l_beta` .

- Compute the vertex neighborhood area `area_B` (using the Voronoi area from `compute_vertex_areas` multiplied by 3 as an approximation).
- Finally $H_v = \text{sum_l_beta}_v / \text{area_B}_v$.

Note: Only interior edges are considered; boundary edges contribute zero (because they have only one adjacent face, and the dihedral angle is undefined).

5. Shannon Entropy

After obtaining λ and H for each vertex, we need to compress them into two scalar features. The paper employs **Shannon entropy**, which measures the "disorder" or "information content" of a distribution. For Alzheimer's patients, the hippocampus shape is more irregular, so the distributions of λ and H tend to be more spread out, resulting in larger entropy values.

5.1 Entropy Formula

For a set of data, divide its value range into M bins, compute the frequency p_i of vertices falling into each bin, then the Shannon entropy is:

$$E = -\sum_{i=1}^{M} p_i \log p_i. \tag{7}$$

5.2 Implementation Details

```
def calculate_surface_entropy(feature_values, bins):
    hist, _ = np.histogram(feature_values, bins=bins)
    probs = hist / np.sum(hist)
    probs = probs[probs > 0]          # remove zero probabilities
    entropy = -np.sum(probs * np.log(probs))
    return entropy
```

Through ablation experiments (Table 4 in the paper), the optimal number of bins was determined: 25 for λ and 20 for H.

The final feature vector is `[entropy_lambda, entropy_H]`, which is fed into an XGBoost classifier.

6. Code Structure and Key Points Recap

- `DiscreteRicciFlow` **class**: Encapsulates metric update, curvature computation, optimization iteration, and planar embedding.
- `main.py` : Executes the complete pipeline, including loading data, identifying the boundary, running Ricci flow, feature extraction, entropy calculation, and saving results.
- **Numerical stability**: Multiple safeguards are added, such as `np.maximum` , `np.clip` , and `np.nan_to_num` , to prevent division by zero or logarithm overflow.
- **Boundary identification**: `find_boundary_vertices` identifies boundary edges by counting edge occurrences (edges appearing only once are boundary edges), and then obtains boundary vertices.

7. Experimental Results and Conclusion

On the ADNI dataset (160 subjects), this method achieved a classification accuracy of **96.88%** (using XGBoost and features from the left hippocampus). This demonstrates the effectiveness of Ricci-flow-based geometric features for Alzheimer's disease diagnosis.

Advantages:

- Fully automatic, requiring no manual landmark annotation.
- Geometric features have clear mathematical meaning and strong interpretability.
- Combining with Shannon entropy effectively captures shape irregularity.

Limitations: Applicable only to triangular meshes and requires pre-determined boundaries.

Workflow Transformation: From "Handcrafted Features" to "Automatic Learning"

Original Workflow:
Extract 3D hippocampal mesh → Align vertex count (down-sampling) → Optimize discrete Ricci flow energy (flatten to 2D) → Compute conformal factor and mean curvature per vertex → Perform binning to calculate Shannon entropy → XGBoost classifier for final output.

New Workflow with DiffusionNet:
Extract 3D hippocampal mesh → Compute Laplacian matrix and initial features → Directly process 3D mesh via DiffusionNet layers → Global pooling → Multi-layer perceptron (MLP) for final output.

Initial Features：

- Raw 3D Coordinates - "xyz"
- Heat Kernel Signatures - "hks"